Jutsu Master
Computer Vision and iOS Game Documentation

Farrell Habibie Putra Haris

July 17, 2026

Contents

Chapter 1

Introduction

1.1 Project Overview

Jutsu Master is a real-time computer vision application that detects Naruto hand signs from live camera input, classifies gestures with a CoreML model, and maps valid sign sequences into interactive jutsu effects and gameplay states on iOS.

The full stack combines:

- dataset preparation and training in Python,
- feature extraction with MediaPipe hand landmarks,
- RandomForest classification exported to CoreML,
- iOS runtime inference, sequence logic, visual effects, and audio.

1.2 Problem Statement

Hand-sign recognition in real-world camera conditions is difficult because of partial visibility, one-hand versus two-hand transitions, and pose variation between users. The project targets a robust unified model that can operate even when only one hand is visible.

1.3 Technical Objectives

- Build a single feature representation for one-hand and two-hand gestures.
- Train a deployable classifier for 12 hand-sign classes.
- Export and integrate the model into an iOS app pipeline.
- Implement sequence-based jutsu detection and game logic.
- Deliver a playable experience across free, tutorial, speed, and battle modes.

1.4 Unified Feature Design

Each hand contributes 21 landmarks with x, y, z coordinates.

$$d = 2 \times 21 \times 3 = 126$$

The model input vector is:

$$\mathbf{x} = [\mathbf{h}_{\text{left}}, \mathbf{h}_{\text{right}}] \in \mathbb{R}^{126}$$

If only one hand is detected, the missing side is filled with 63 zeros. This preserves a fixed input dimension while retaining one-hand usability.

1.5 End-to-End Runtime Flow

1. Capture a camera frame.
2. Detect hand landmarks (MediaPipe or Vision fallback).
3. Normalize and merge features into a 126-dimensional vector.
4. Run CoreML gesture inference.
5. Apply hold logic and sequence tracking.
6. Trigger jutsu effect, audio, and mode-specific game state updates.

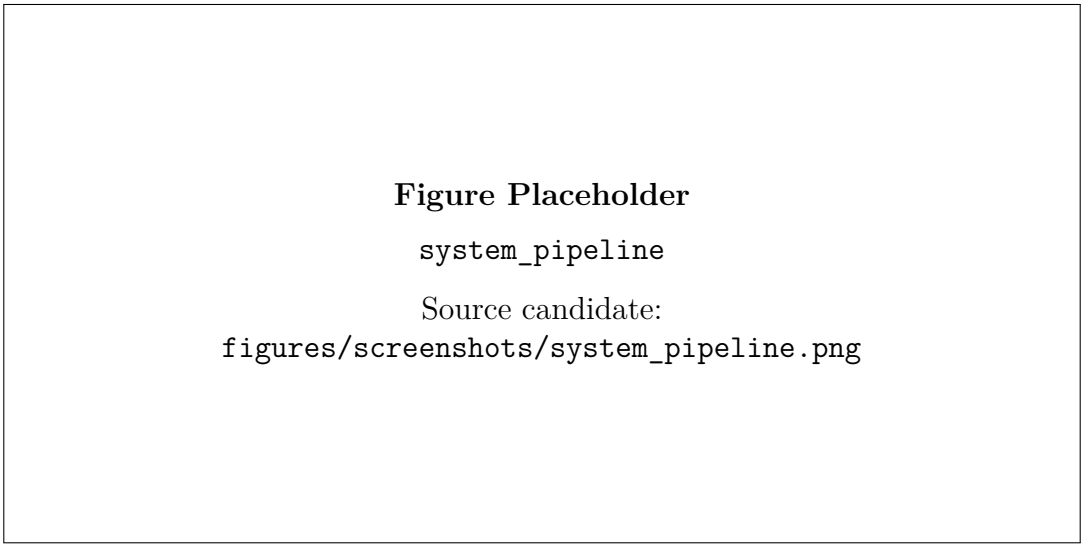

Figure 1.1: System pipeline placeholder. Replace with a final architecture diagram or app pipeline screenshot.

1.6 Document Scope

This report documents dataset preparation, model training, iOS architecture, gameplay logic, measured results, and current limitations, with figure placeholders for assets and screenshots.

Chapter 2

Dataset

2.1 Source and Annotation Format

The training dataset is based on YOLO-style labels where each annotation stores class index and normalized bounding box coordinates.

Primary dataset configuration:

- Location: `hand_gesture_model/naruto-hand-sign-4/`
- Config file: `hand_gesture_model/naruto-hand-sign-4/data.yaml`
- Number of classes: 12
- Class names: bird, boar, dog, dragon, hare, horse, monkey, ox, ram, rat, snake, tiger

2.2 Data Splits

The dataset uses train, validation, and test folder splits:

- `train/images` and `train/labels`
- `valid/images` and `valid/labels`
- `test/images` and `test/labels`

2.3 Class Taxonomy

Table 2.1: Hand-sign classes from dataset metadata

Class count	12
Sign labels	bird, boar, dog, dragon, hare, horse, monkey, ox, ram, rat, snake, tiger

2.4 Class Distribution (Exported Unified Dataset)

Table 2.2: Class counts from `class_distribution_unified.csv`

Class	Count	Class	Count
bird	124	monkey	562
boar	105	ox	302
dog	368	ram	370
dragon	367	rat	259
hare	118	snake	280
horse	455	tiger	651

2.5 Label Alias Note

In iOS gameplay logic, *hare* is normalized to *rabbit* for asset naming and sequence consistency. This alias handling should be kept explicit in deployment documentation.

2.6 Attached Asset Placeholders

The following placeholders correspond to attached sign and reference assets.

Figure Placeholder

`asset_handsign_bird`

Source candidate:
`../naruto_app/naruto_app/handsign_image/Bird.webp`

Figure 2.1: Bird hand-sign reference asset placeholder.

Figure Placeholder

`asset_handsign_dragon`

Source candidate:
`../naruto_app/naruto_app/handsign_image/Dragon.webp`

Figure 2.2: Dragon hand-sign reference asset placeholder.

Figure Placeholder

`asset_handsign_tiger`

Source candidate:
`../naruto_app/naruto_app/handsign_image/Tiger.webp`

Figure 2.3: Tiger hand-sign reference asset placeholder.

Figure Placeholder

`asset_handsign_rat`

Source candidate:
`../naruto_app/naruto_app/handsign_image/Rat.png`

Figure 2.4: Rat hand-sign reference asset placeholder.

Chapter 3

Model Training

3.1 Notebook Scope

Training is implemented in:

- `hand_gesture_model/train2.ipynb`

This notebook builds a unified one-hand and two-hand classifier with fixed 126-feature input vectors.

3.2 Preprocessing and Feature Extraction

For each labeled bounding box:

1. Read YOLO box and crop hand region.
2. Detect up to two hands with MediaPipe.
3. Normalize each hand relative to wrist landmark.
4. Scale landmarks by the maximum 2D norm.
5. Assign features to left and right slots.
6. Zero-pad missing hand slot when only one hand is present.

Normalization equation per hand:

$$\hat{\mathbf{p}}_i = \mathbf{p}_i - \mathbf{p}_0, \quad s = \max_j \|\hat{\mathbf{p}}_{j,xy}\|_2, \quad \tilde{\mathbf{p}}_i = \frac{\hat{\mathbf{p}}_i}{\max(s, 10^{-6})}$$

Final feature vector:

$$\mathbf{x} = [\tilde{\mathbf{p}}_{L,1..21}, \tilde{\mathbf{p}}_{R,1..21}] \in \mathbb{R}^{126}$$

3.3 Training Configuration

- Model: RandomForestClassifier
- Trees: 400
- Class balancing: `balanced_subsample`
- Split (v1, `train2.ipynb`): 80:20 stratified train-test over pooled samples
- Split (v2, `train_unified_v2.py`): original Roboflow train/valid/test folders
- Random seed: 42

3.4 Evaluation Protocol v2 (Leak-Free Split)

The v1 protocol pooled samples from all Roboflow folders and re-split them randomly. Because the source dataset contains bursts of near-identical frames of the same person performing the same sign, a random split can place sibling frames on both sides of the train/test boundary and inflate the estimate.

The v2 protocol (`hand_gesture_model/train_unified_v2.py`) instead honors the dataset's original split: 2,768 train, 798 validation, and 395 test samples. Model selection uses validation macro-F1 only; the test split is evaluated once for the final report.

3.4.1 Landmark-Space Augmentation Study

Two augmentation variants were evaluated on the training split only:

- **Mirror** (2×): negate x after wrist-centering and swap the left/right hand blocks.
- **Full** (4×): mirror plus two rotations of (x, y) around the wrist by $\pm(5\text{–}15)^\circ$ with Gaussian jitter ($\sigma = 0.01$).

Neither variant improved validation macro-F1 beyond the selection margin (0.005), and both *reduced* held-out test performance (mirror: 96.46%; full: 96.96%; no augmentation: 97.47%). The mirror result is notable: hand signs are chirality-sensitive (which hand crosses on top is class-relevant), so mirrored copies inject label noise. The deployed v2 model therefore uses the clean, non-augmented training set.

3.5 Exported Artifacts

Unified pipeline outputs are written to:

- `hand_gesture_model/naruto-hand-sign-4/phase1_outputs_unified/hand_landmarks_datase`
- `hand_gesture_model/naruto-hand-sign-4/phase1_outputs_unified/X_landmarks_unified.n`
- `hand_gesture_model/naruto-hand-sign-4/phase1_outputs_unified/y_labels_unified.npy`
- `hand_gesture_model/naruto-hand-sign-4/phase1_outputs_unified/confusion_matrix_unif`
- `hand_gesture_model/naruto-hand-sign-4/phase1_outputs_unified/hand_gesture_rf_unifi`

- `hand_gesture_model/naruto-hand-sign-4/phase1_outputs_unified/training_summary_uni`

The v2 pipeline additionally writes to `phase1_outputs_unified_v2/`:

- `hand_gesture_rf_unified.mlmodel` (deployed to the iOS app)
- `classification_report_v2.txt` (per-class precision/recall/F1)
- `confusion_matrix_v2.csv`
- `training_summary_v2.json`

3.6 Evaluation Metrics

Table 3.1: Unified model extraction and evaluation metrics (v2, leak-free split)

Metric	Value
Usable samples	3961
One-hand samples	3289 (83.03%)
Two-hand samples	672 (16.97%)
Skipped images	2181
Skipped no hands	2177
Feature dimension	126
Split	Roboflow train/valid/test (2768/798/395)
Test accuracy (v2, clean split)	**97.47%**
Test macro-F1 (v2, clean split)	0.9709
Reference: v1 pooled random 80:20 accuracy	96.85%
CoreML export	Success (hand_gesture_rf_unified.mlmodel)

3.7 Confusion Matrix Notes

The exported confusion matrix shows strong diagonal dominance, with minor confusion among visually similar signs (for example, dragon vs snake, and ram vs tiger under some samples).

Figure Placeholder

`confusion_matrix_placeholder`

Source candidate:
`figures/screenshots/confusion_matrix_heatmap.png`

Figure 3.1: Confusion matrix heatmap placeholder generated from CSV metrics.

Figure Placeholder

`training_notebook_outputs`

Source candidate:
`figures/screenshots/training_notebook_outputs.png`

Figure 3.2: Notebook output screenshot placeholder for extraction summary and training logs.

Chapter 4

iOS Architecture

4.1 Application Structure

The iOS app is implemented in:

- `naruto_app/naruto_app/`

  Top-level flow:

- `naruto_appApp.swift` launches `ContentView.swift`.
- Navigation routes users from Home to Mode Setup and then into game views.
- `CameraGameView.swift` handles free, speed, and tutorial modes.
- `BattleGameView.swift` combines camera inference with battle simulation.

4.2 Core Runtime Managers

Table 4.1: Manager responsibilities in the runtime pipeline

Component	Responsibility
CameraManager	Camera permission, session setup, frame streaming, orientation and mirroring management.
HandLandmarkDetector	Hand landmark extraction with MediaPipe Tasks and Vision fallback.
FaceDirectionEstimator	Mouth and face direction estimation for fireball and burning ash effects.
GestureRecognizer	Feature preprocessing, CoreML model inference, score formatting, top-k output.
JutsuManager	Hold logic, sequence tracking, mode-specific trigger decisions, effect state memory.

4.3 Model and Backend Fallback Strategy

- Gesture model loading uses the unified 126-feature model exclusively; legacy one-hand and two-hand models were removed because their feature layouts are incom-

patible and a silent fallback would corrupt predictions. A missing or unloadable model now fails loudly (assertion + fault log).

- The recognizer records a rolling average of end-to-end pipeline latency (landmarks → features → CoreML) and logs it every 120 frames via `os.log`, so real-time claims are measured.
- Landmark detection attempts MediaPipe runtime task models and falls back to Vision requests if unavailable.
- Face direction estimation similarly uses MediaPipe face mesh first with Vision fallback.

4.4 Frame-to-Action Pipeline

1. Camera delivers a sample buffer.
2. Hand and face landmarks are extracted.
3. Landmark vectors are canonicalized for mirror and orientation consistency.
4. Classifier predicts sign label and confidence.
5. JutsuManager processes hold and sequence state.
6. Active jutsu state updates overlays, particles, sounds, and game counters.

4.5 Rendering Stack

- `CameraView.swift` overlays skeletons and effects above AVCapture preview.
- CAEmitterLayer handles fire, lightning, and ash particle systems.
- SpriteKit scenes handle water dragon and kuchiyose summon animations.
- Overlay UI components display target sign guidance and progress strips.

Figure Placeholder

`ios_architecture_diagram`

Source candidate:
`figures/screenshots/ios_architecture_diagram.png`

Figure 4.1: iOS architecture diagram placeholder.

Figure Placeholder

`camera_overlay_runtime`

Source candidate:
`figures/screenshots/runtime_prediction.png`

Figure 4.2: Runtime camera overlay screenshot placeholder.

Chapter 5

Gameplay

5.1 Game Modes

The app supports four user modes:

- Battle: defend and counterattack against Sasuke with HP and chakra systems.
- Free: trigger any supported jutsu without fixed target constraints.
- Speed: complete a selected jutsu sequence as fast as possible.
- Tutorial: follow guided signs for a selected target jutsu.

5.2 Sign Sequence Logic

Each recognized sign must be held before commitment, then matched against jutsu sequences. The same recognized signs can produce different outcomes depending on mode and context.

Table 5.1: Jutsu-to-sign mapping used by game logic

Jutsu	Sign sequence	Count
Fire Style: Burning Ash	dragon → ox → dog → horse	4
Fire Style: Fireball Jutsu	horse → snake → monkey → boar → horse	5
Fire	bird → snake	2
Kuchiyose no Jutsu	boar → horse → monkey → bird	4
Chidori	ox → monkey	2
Rasengan	monkey → bird	2
Water Style: Water Dragon Bullet	monkey → boar → bird → ox → horse → bird → dog → snake	8
Wind Style: Rasengan	horse → monkey	2

5.3 Battle Mechanics

Battle mode adds a state machine around CV inference:

1. Enemy attack phase selects an incoming jutsu.
2. Player defend phase expects a specific counter jutsu sequence.
3. Player attack phase allows offensive jutsu launches.
4. Round end restores partial chakra and advances battle pacing.

Counter mapping examples:

- Fireball or Burning Ash is countered by Water Dragon.
- Lightning is countered by Rasengan.

5.4 Visual and Audio Feedback

- Overlay guidance shows current target sign and sequence progression.
- Particle and scene effects represent each jutsu style.
- Audio loops adapt volume and rate for active effects such as Rasengan and Wind.

5.5 Attached Asset Placeholders

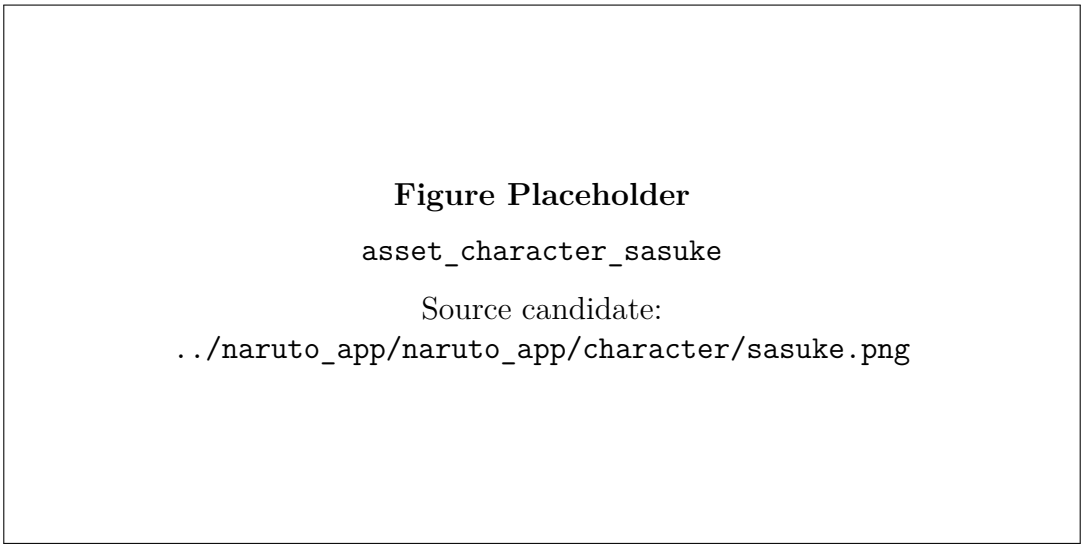

Figure 5.1: Sasuke character asset placeholder used in battle mode.

Figure Placeholder

`asset_reference_rasengan`

Source candidate:
`../naruto_app/naruto_app/image_reference/rasengan.jpg`

Figure 5.2: Rasengan visual reference asset placeholder.

Figure Placeholder

`asset_kuchiyose_gamabunta`

Source candidate:
`../naruto_app/naruto_app/kuchiyose/gamabunta.jpg`

Figure 5.3: Gamabunta summon asset placeholder.

Figure Placeholder

`asset_kuchiyose_manda`

Source candidate:
`../naruto_app/naruto_app/kuchiyose/manda.jpg`

Figure 5.4: Manda summon asset placeholder.

5.6 App Screenshot Placeholders

Figure Placeholder

`screenshot_home_view`

Source candidate:
`figures/screenshots/home_view.png`

Figure 5.5: Home view screenshot placeholder.

Figure Placeholder

`screenshot_mode_setup`

Source candidate:
`figures/screenshots/mode_setup.png`

Figure 5.6: Mode setup screenshot placeholder.

Figure Placeholder

`screenshot_tutorial_overlay`

Source candidate:
`figures/screenshots/tutorial_overlay.png`

Figure 5.7: Tutorial overlay screenshot placeholder.

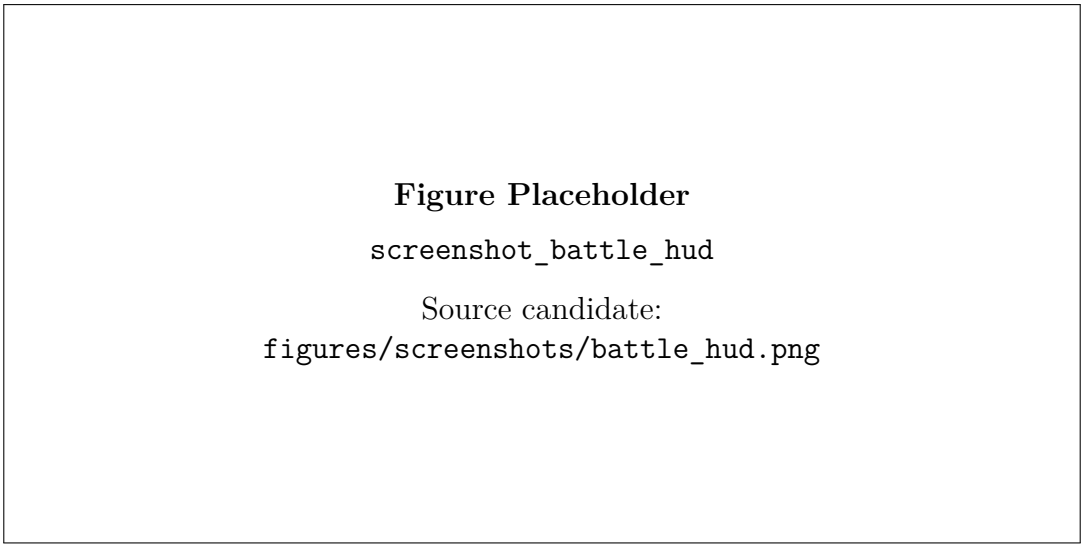

Figure 5.8: Battle HUD screenshot placeholder.

Chapter 6

Results

6.1 Quantitative Summary

The unified model achieved strong held-out test accuracy under the v2 leak-free evaluation protocol and a successful CoreML export for on-device use.

- Test accuracy (v2, Roboflow test split): 97.47%
- Test macro-F1 (v2): 0.9709
- Reference v1 (pooled random 80:20): 96.85%
- Feature dimension: 126
- Usable samples: 3961 (train 2768 / valid 798 / test 395)
- One-hand coverage: 83.03%; two-hand coverage: 16.97%

6.2 Model Behavior Observations

- Most classes are separated reliably under clean lighting and stable framing; horse, monkey, dragon, and bird reach 98–100% F1 on the held-out test split.
- The weakest classes are snake (F1 0.92) and boar (F1 0.94), consistent with visually similar finger formations.
- One-hand fallback through zero padding improves practical robustness.
- Landmark-space augmentation did not help: mirroring reduced test accuracy to 96.46%, confirming that hand signs are chirality-sensitive, and rotation augmentation also degraded performance (96.96%). The deployed model uses the clean training set.

6.3 iOS Runtime Observations

- Real-time inference supports responsive sequence gameplay.
- Combined hand and face cues improve fire-style effect direction logic.
- Battle mode adds meaningful interaction through defend and attack windows.

6.4 Runtime Screenshot Placeholders

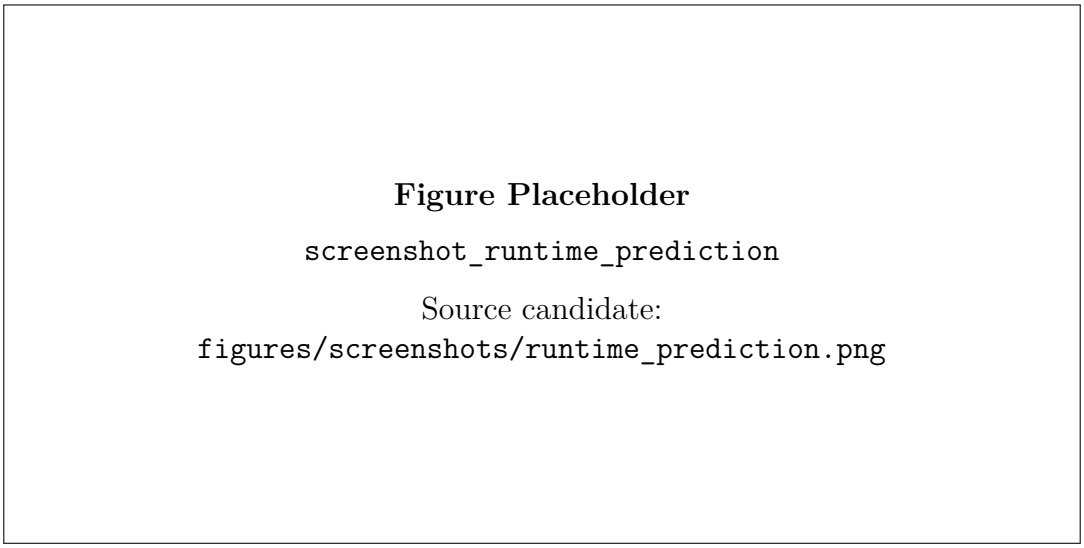

Figure 6.1: Runtime sign prediction screenshot placeholder.

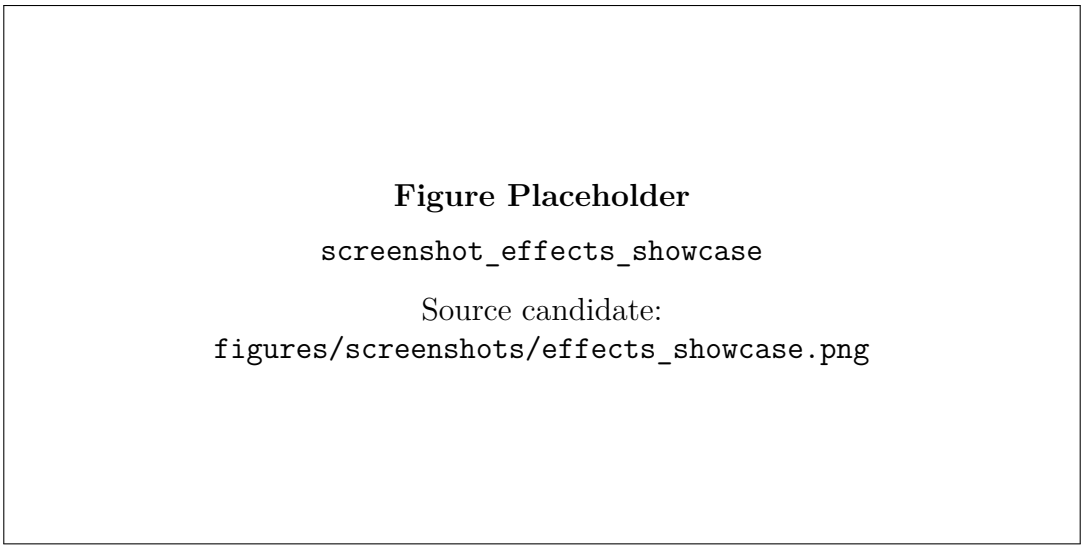

Figure 6.2: Effects showcase screenshot placeholder (fire, lightning, rasengan, water dragon, kuchiyose).

Figure Placeholder

`screenshot_battle_sequence`

Source candidate:
`figures/screenshots/battle_sequence.png`

Figure 6.3: Battle sequence progress screenshot placeholder.

Chapter 7

Limitations and Future Work

7.1 Current Limitations

7.1.1 Dataset and Labeling

- Class imbalance remains visible (for example, tiger vs boar support counts).
- Background and lighting variation may still be limited compared to production use.
- Label naming mismatch (hare vs rabbit aliasing) adds integration complexity.

7.1.2 Model and Inference

- RandomForest operates on per-frame static features and does not model temporal dynamics directly.
- Confidence thresholds require manual tuning per runtime environment.
- Similar hand poses can still produce occasional confusion.

7.1.3 iOS Runtime and UX

- Performance and stability may vary by device generation and camera quality.
- Effects and tracking quality are sensitive to fast motion blur and partial occlusion.
- Guidance timing and hold duration messaging should remain synchronized with logic constants.

7.2 Addressed Since v1

- Leak-free evaluation protocol honoring the Roboflow train/valid/test split (`train_unified_v2.py`) with per-class reports and a documented augmentation study.
- Unified-only CoreML model loading with loud failure instead of silent fallback to incompatible legacy models.
- Rolling inference-latency instrumentation in `GestureRecognizer`.

- Clock injection in `JutsuManager.processCandidate` and a 14-test SwiftPM suite (`swift_tests/`) covering hold-to-commit, flicker rejection, label aliasing, sequence triggers, time limits, and battle-mode counters.

7.3 Resolved Issues

- **Kuchiyose unreachable in free mode (fixed via trigger deferral)**: its sequence (boar, horse, monkey, bird) contains wind's (horse, monkey) as a substring, so wind used to fire at the monkey step and clear the sign history. The fix defers a completed short jutsu for a 1.8 s grace window whenever the sign history is still a live prefix of a longer sequence that contains it: the short jutsu fires when the window expires (player stopped signing) and is discarded when the longer jutsu completes. Standalone short sequences are unaffected. Covered by three unit tests in `swift_tests/`.

7.4 Versus Mode (Two Players, One Camera)

A new mode splits the camera frame down the middle: the hand landmarker tracks up to four hands, hands are grouped by which half of the frame they occupy, and each side receives an independent 126-feature prediction from the same unified CoreML model. Both players sign simultaneously; a completed sequence launches that jutsu at the opponent (damage scales with sequence length, 12–34 HP), and casting the elemental counter before an incoming jutsu lands blocks it. First player to zero HP loses.

7.5 Tutorial Kuchiyose Fix

The kuchiyose time limit (previously 4.5 s) and its instant wrong-sign reset are speed-mode challenge rules; they also applied in the tutorial, where holding four signs plus reading the on-screen guide almost always exceeded the window — causing the reported "always fails at the last sign" behavior. The tutorial now learns without the timer and with the standard 2 s wrong-sign grace period, while speed mode keeps both rules with the limit relaxed to 6 s.

7.6 Battle Mode v2

The battle system was reworked for interactivity:

- Enemy projectiles now deal chip damage when they cross the player's zone during the defend phase, making defense time-critical rather than a passive pass/fail check.
- Defense grading: countering within the first 3 s of the defend phase scores a *Perfect Block* (+10 chakra).
- Attack-phase combos: chaining jutsu within a 4 s window raises a damage multiplier (up to 1.6×) with on-screen combo feedback.
- Difficulty scaling: enemy barrage rate increases 8% per round, defend windows shorten, and failed-defense damage grows.

- Feedback layer: impact-burst rings, enemy hit-flash, screen shake on player damage, distinct victory/defeat overlays, and haptic feedback for sign commits, triggers, blocks, and hits.

7.7 Recommended Improvements

1. Add temporal modeling (sequence-aware classifier or lightweight recurrent model).
2. Expand training data with multi-user and multi-environment captures.
3. Automate threshold calibration and per-class confidence analysis.
4. Add model versioning and reproducible experiment logging.
5. Build an automated evaluation suite for on-device regression testing.
6. Integrate screenshot and telemetry capture for documentation updates.

7.8 Documentation Maintenance Plan

- Update metrics tables after each model retrain.
- Replace placeholders with latest app screenshots before submission.
- Keep jutsu-sequence table aligned with `AppModels.swift` source changes.

Appendix A

Expected Screenshot Files

Place screenshots in the following paths and recompile:

- `figures/screenshots/home_view.png`
- `figures/screenshots/mode_setup.png`
- `figures/screenshots/tutorial_overlay.png`
- `figures/screenshots/battle_hud.png`
- `figures/screenshots/runtime_prediction.png`
- `figures/screenshots/effects_showcase.png`
- `figures/screenshots/confusion_matrix_heatmap.png`